ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ
TECHNICAL UNIVERSITY OF CRETE

Electrical and Computer Engineering

Number Theory & Cryptography
Assignment 3

Course Instructor

Prof. Georgios Karystinos

Assignment Supervisor

Ioannis-Leonidas Steiakakis
isteiakakis@tuc.gr

Assignment Instructions

For the programming parts of the assignment, the accepted programming language is Python 3 (as `.py` files, not Jupyter notebooks).

For tasks requiring code implementation, name the script file for each task appropriatelly, as shown in the following tree diagram depicting the file structure in your deliverable.

For a task requiring code implementation, you may copy-paste a script from a previous task and use it as a starting point.

Try to follow good programming practices.

- You may refactor code into functions for modularity and logical abstraction for a cleaner code in the main script file of a task.
- Please use appropriate comments: clear and explanatory, but not overly verbose or unnecessary (e.g., avoid commenting on self-explanatory function calls and easily understandable lines of code).

In code simulation tasks, the indicative values given for parameters are just indicative. Do not hardcode them. Your code should be parameterized by those parameters and should work for other (appropriate) values, too.

You may use an immediate result (i.e., without proof) directly from the course notes, provided you appropriately reference[1] it, of course. Make sure to adjust the notation to match that of the assignment prompt and your deliverable report.

Important! Your solutions must strictly follow the theory from course notes. Any non-trivial concepts, whether in theory or code, must be thoroughly explained. For theoretical aspects, provide clear reasoning and justification. For code, ensure that your implementation is well-commented and reflects the underlying theory. Theoretical ideas applied to your code should also be clearly justified and well-documented. Lack of proper explanation may lead to point deductions.

Also important! Your deliverable must follow the structure shown below. `etc/` is a directory for any other file you may want to include. `util.py` is a file with functions implemented by you which can be used in the task scripts by importing it (`import util`). `task*.py` files are your implementations of each task; of course an implementation can be moved in a function in `util.py` and just demonstrated in the task script. They must run as `` `python3 task`⟨*id*⟩⟨*suffix*⟩`.py` `` (no extra command line arguments), where *id* is the task ID and *suffix* is optional for tasks with multiple script files.

[1] That is, clearly indicate the source of the result; be precise (e.g., equation number, theorem number, etc).

```
report.pdf
src/
|-- etc/
|   |-- ...
|-- util.py
|-- task1c.py
|-- task1d.py
|-- task2e.py
|-- task2f.py
|-- task2g.py
|-- task2h.py
|-- task2i.py
```

Preliminaries

Definitions and Instrustions for the Assignment

- $\mathcal{T}_n \triangleq \{a \in \{1, \ldots, n\} : (a, n) = 1\}$ for any $n \geq 1$.
- $s \xleftarrow{\$} S$ denotes sampling an element s uniformly at random from the set S.
- To sample from a proper subset of $\mathbb{Z}_n$, first sample from $\mathbb{Z}_n$ (using `randbelow` or `randbits` mentioned below) and then apply rejection sampling.
  - Except when sampling primes, where you may use `getPrime` mentioned below.

Rules to Follow

For algebraic operations, you must use only Python operators, such as:

- Arithmetic operators: `+`, `-`, `*`, `/`, `//`, `%`, `**`
- Bitwise operators: `&`, `|`, `^`, `~`, `<<`, `>>`
- And, of course, their assignment variants, e.g., `+=`, `-=`, ...

Also, the following built-in functions are allowed: `abs`, `sum`, `min`, `max`

You are not allowed to import anything other than your modules and the following:

```
from secrets import randbelow, randbits
from Crypto.Util.number import getPrime, isPrime
```

Modules like `os`, `shutil`, `sys`, etc, are also acceptable.

Lists and dictionaries are allowed.

The restriction on operators and modules is intended to avoid using ready-made implementations in cases where the functionality must be implemented by you. If you wish to use something that you believe does not violate this, but is not listed above, you may ask for clarification.

Contents

1 Abstracting Operations Beyond Numbers **1**

Task 1.a (10%) 1

Task 1.b (10%) 1

Task 1.c (5%) 2

Task 1.d (5%) 2

2 Polynomial arithmetic **3**

Task 2.a (2%) 3

Task 2.b (2%) 3

Task 2.c (3%) 3

Task 2.d (12%) 3

Task 2.e (12%) 3

Task 2.f (12%) 3

Task 2.g (12%) 4

Task 2.h (7%) 4

Task 2.i (8%) 4

1 Abstracting Operations Beyond Numbers

Let a set G and the binary operation $* : G \times G \to G$.

Let the following grid and an agent which starts from the square "a".

d	e	f
a	b	c

G is the set of actions, two of which are the actions "move to the square to the right" and "move to the square above". The operation $*$ combines two actions and returns another action.

If at any point the agent steps out of the grid, then it goes to the corresponding square as if the grid was repeated.

d	e	f	d	e	f
a	b	c	a	b	c
d	e	f	d	e	f
a	b	c	a	b	c

Task 1.a (10%)

Derive the truth table and check which properties of the commutative group definition hold.

Task 1.b (10%)

Same as Task 1.a, but now assume that if at any point the agent steps out of the grid, then it goes to square "a".

For the following tasks, we define the One-Time Pad (OTP) cryptosystem for the structure $\langle G, * \rangle$.

$$\text{Enc}_k(m) = m * k$$
$$\text{Dec}_k(c) = c * k'$$

where $m, c, k, k' \in G$, $c = \text{Enc}_k(m)$, $k * k' = e$ for $e \in G$ being the identity element.

For a message containing consecutive elements from G, we apply OTP to each element using a new random key.

Task 1.c (5%)

Implement in code the OTP cryptosystem (encryption and decryption) for a message in G^{100} for the case of Task 1.a. Does it work correctly? Explain.

Task 1.d (5%)

Same as Task 1.c, but for the case of Task 1.b.

2 Polynomial arithmetic

For the following three tasks, let $\langle \mathbb{Z}_2, +, \cdot \rangle$ where the operations $+, \cdot$ are defined as the usual addition and multiplication (respectively) modulo 2.

The polynomial operations $\oplus, \odot$ are defined the same way as in course notes.

Task 2.a (2%)

Is the structure $\langle \mathbb{Z}_2[x]/(x^3+1), \oplus, \odot \rangle$ a field? Justify your answer.

Task 2.b (2%)

Is the structure $\langle \mathbb{Z}_2[x]/(x^3+x^2+1), \oplus, \odot \rangle$ a field? Justify your answer.

Task 2.c (3%)

Derive (manually, without using software) the truth table for Task 2.a and Task 2.b. Describe how to find the multiplicative inverse of each element using the truth table.

Task 2.d (12%)

Derive (analytically, by hand yet *algorithmically*) the multiplicative inverse of x^2+1 in $\langle \mathbb{Z}_2[x]/(x^3+x^2+1), \oplus, \odot \rangle$. (Do not use the previous truth table as a lookup table.)

Note: It is also part of the task to derive the algorithm before applying it.

Task 2.e (12%)

Let $\langle R, +, \cdot \rangle$ be a ring. Implement in code a function that computes the addition (i.e., $\oplus$) between two polynomials in $R[x]$. Same for multiplication (i.e., $\odot$). Same for the additive inverse of a polynomial in $R[x]$.

The ring $\langle R, +, \cdot \rangle$ must not be hardcoded; your implementation must be parameterized by it.

Task 2.f (12%)

Let $\langle F, +, \cdot \rangle$ be a field. Implement in code a function that computes the quotient and the remainder of the polynomial division between two polynomials in $F[x]$.

As above, the field $\langle F, +, \cdot \rangle$ must not be hardcoded; your implementation must be parameterized by it.

Task 2.g (12%)

Let $\langle F, +, \cdot \rangle$ be a field and $f(x) \in F[x]$. Implement in code a function that computes the multiplicative inverse of a polynomial in $F[x]/f(x)$.

As above, the field $\langle F, +, \cdot \rangle$ must not be hardcoded; your implementation must be parameterized by it.

State which functions $f(x)$ are appropriate for the above task (your code does not need to perform this check).

$f(x)$ is part of the function parameters, do not hardcode it.

For the following tasks regarding implementation of some AES steps, you should be able to use your implementations from the previous tasks.

Task 2.h (7%)

Implement in code a function that performs the AES SubBytes step.

Task 2.i (8%)

Implement in code a function that performs the AES MixColumns step.